

Files and Streams

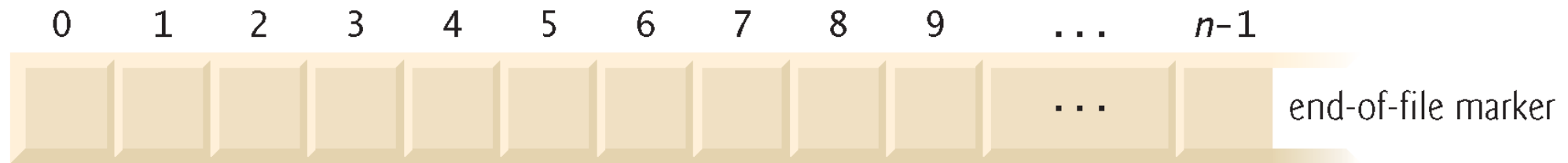
Visual C# How to Program, 6/e

Introduction

- ▶ Variables and arrays offer only temporary storage of data—the data is lost when a local variable “goes out of scope” or when the program terminates.
- ▶ By contrast, files (and databases) are used for long-term retention of large amounts of data, even after the program that created the data terminates.
- ▶ Data maintained in files often is called persistent data.
- ▶ Data maintained in files often is called **persistent data**. Computers store files on **secondary storage devices**, such as hard drives, solid-state drives, flash drives, DVDs and tapes

Files and Streams

- ▶ C# views each file as a sequential stream of bytes.
- ▶ Each file ends either with an end-of-file marker or at a specific byte number that's recorded in a system-maintained administrative data structure.
- ▶ When a file is opened, an object is created and a stream is associated with the object.



Files and Streams (cont.)

- ▶ When a console app executes, the runtime environment creates three stream objects that are accessible via properties `Console.Out`, `Console.In` and `Console.Error`, respectively.
- ▶ These objects use streams to facilitate communication between a program and a particular file or device.
- ▶ `Console.In` refers to the standard input stream object, which enables a program to input data from the keyboard.
- ▶ `Console.Out` refers to the standard output stream object, which enables a program to output data to the screen.
- ▶ `Console.Error` refers to the standard error stream object, which enables a program to output error messages to the screen.

Files and Streams (cont.)

- ▶ Abstract class `Stream` provides functionality for representing streams as bytes. Classes `FileStream`, `MemoryStream` and `BufferedStream` (all from namespace `System.IO`) inherit from class `Stream`.
- ▶ Class `FileStream` can be used to write data to and read data from files.
- ▶ Class `MemoryStream` enables the transfer of data directly to and from memory—this is much faster than reading from and writing to external devices.
- ▶ Class `BufferedStream` uses buffering to transfer data to or from a stream.

Files and Streams (cont.)

- ▶ The `System.IO` namespace includes stream classes such as `StreamReader` (for text input from a file), `StreamWriter` (for text output to a file) and `FileStream` (for both input from and output to a file).
- ▶ These stream classes inherit from abstract classes `TextReader`, `TextWriter` and `Stream`, respectively.
- ▶ Properties `Console.In` and `Console.Out` are of type `TextReader` and `TextWriter`, respectively.
- ▶ The system creates objects of `TextReader` and `TextWriter` derived classes to initialize `Console` properties `Console.In` and `Console.Out`.

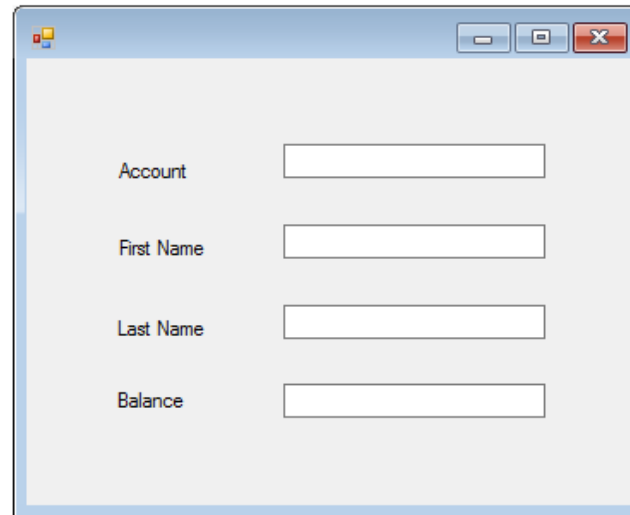
Creating a Sequential-Access Text File

- ▶ C# imposes no structure on files.
- ▶ Thus, the concept of a “record” does not exist in C# files.
- ▶ This means that you must structure files to meet the requirements of your apps.

Creating a Sequential-Access Text File (cont.)

Class BankUIForm

- ▶ The following examples demonstrate file processing in a bank-account maintenance app.

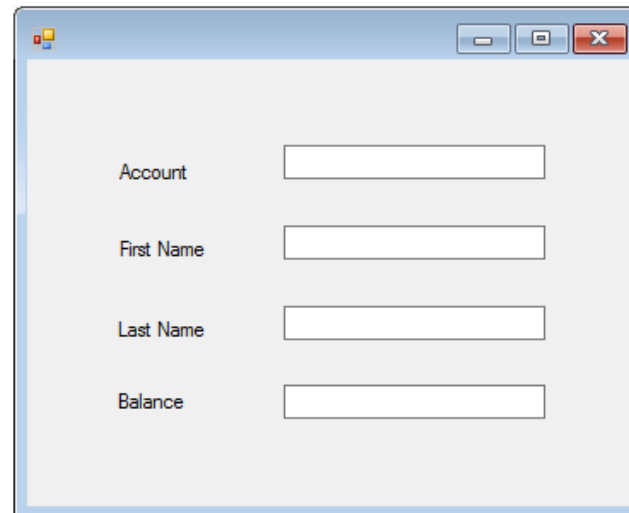
A screenshot of a Windows-style application window titled "BankUIForm". The window has a light gray background and a blue border. It contains four text input fields arranged vertically. The labels for the fields are "Account", "First Name", "Last Name", and "Balance", positioned to the left of each input box. The input boxes are empty and have a thin gray border. The window's title bar includes standard Windows controls: a minimize button, a maximize button, and a close button.

```
1  // Fig. 17.2: BankUIForm.cs
2  // A reusable Windows Form for the examples in this chapter.
3  using System;
4  using System.Windows.Forms;
5
6  namespace BankLibrary
7  {
8      public partial class BankUIForm : Form
9      {
10         protected int TextBoxCount { get; set; } = 4; // number of TextBoxes
11
12         // enumeration constants specify TextBox indices
13         public enum TextBoxIndices {Account, First, Last, Balance}
14
15         // parameterless constructor
16         public BankUIForm()
17         {
18             InitializeComponent();
19         }
```

```
20
21 // clear all TextBoxes
22 public void ClearTextBoxes()
23 {
24     // iterate through every Control on form
25     foreach (Control guiControl in Controls)
26     {
27         // if Control is TextBox, clear it
28         (guiControl as TextBox)?.Clear();
29     }
30 }
31
```

```
32      // set text box values to string-array values
33      public void SetTextBoxValues(string[] values)
34      {
35          // determine whether string array has correct length
36          if (values.Length != TextBoxCount)
37          {
38              // throw exception if not correct length
39              throw (new ArgumentException(
40                  $"There must be {TextBoxCount} strings in the array",
41                  nameof(values)));
42          }
43          else // set array values if array has correct length
44          {
45              // set array values to TextBox values
46              accountTextBox.Text = values[(int) TextBoxIndices.Account];
47              firstNameTextBox.Text = values[(int) TextBoxIndices.First];
48              lastNameTextBox.Text = values[(int) TextBoxIndices.Last];
49              balanceTextBox.Text = values[(int) TextBoxIndices.Balance];
50          }
51      }
```

```
52
53 // return TextBox values as string array
54 public string[] GetTextBoxValues()
55 {
56     return new string[] {
57         accountTextBox.Text, firstNameTextBox.Text,
58         lastNameTextBox.Text, balanceTextBox.Text};
59     }
60 }
61 }
```



Account

First Name

Last Name

Balance

Creating a Sequential-Access Text File (cont.)

- ▶ Using visual inheritance, you can extend this class to create the GUIs for several examples.
- ▶ Recall that to reuse class `BankUIForm`, you must compile the GUI into a class library, then add a reference to the new class library in each project that will reuse it.
- ▶ This library (`BankLibrary`) is provided with the code for this example.
- ▶ You might need to re-add the references to this library in our examples when you copy them to your system, since the library most likely will reside in a different location on your system.

17.3 Creating a Sequential-Access Text File (cont.)

Class Record

- ▶ Record class is used for maintaining the information in each record that's written to or read from a file.
- ▶ This class also belongs to the BankLibrary DLL, so it's located in the same project as class BankUIForm.

```
1  // Fig. 17.3: Record.cs
2  // Class that represents a data record.
3  namespace BankLibrary
4  {
5      public class Record
6      {
7          public int Account { get; set; }
8          public string FirstName { get; set; }
9          public string LastName { get; set; }
10         public decimal Balance { get; set; }
11
12         // parameterless constructor sets members to default values
13         public Record() : this(0, string.Empty, string.Empty, 0M) { }
14
```

```
15      // overloaded constructor sets members to parameter values
16      public Record(int account, string firstName,
17                    string lastName, decimal balance)
18      {
19          Account = account;
20          FirstName = firstName;
21          LastName = lastName;
22          Balance = balance;
23      }
24  }
25 }
```

Creating a Sequential-Access Text File (cont.)

Using a Character Stream to Create an Output File

- ▶ Class `CreateFileForm` uses instances of class `Record` to create a sequential-access file that might be used in an accounts-receivable system—i.e., a program that organizes data regarding money owed by a company's credit clients.
- ▶ For each client, the program obtains an account number and the client's first name, last name and balance (i.e., the amount of money that the client owes to the company for previously received goods and services).
- ▶ The data obtained for each client constitutes a record for that client.

Creating a Sequential-Access Text File (cont.)

- ▶ In this app, the account number is used as the record key—files are created and maintained in account-number order.
- ▶ This program assumes that the user enters records in account-number order. However, a comprehensive accounts-receivable system would provide a sorting capability, so the user could enter the records in any order.

```
1  // Fig. 17.4: CreateFileForm.cs
2  // Creating a sequential-access file.
3  using System;
4  using System.Windows.Forms;
5  using System.IO;
6  using BankLibrary;
7
8  namespace CreateFile
9  {
10     public partial class CreateFileForm : BankUIForm
11     {
12         private StreamWriter fileWriter; // writes data to text file
13
14         // parameterless constructor
15         public CreateFileForm()
16         {
17             InitializeComponent();
18         }
19     }
```

```
20 // event handler for Save Button
21 private void saveButton_Click(object sender, EventArgs e)
22 {
23     // create and show dialog box enabling user to save file
24     DialogResult result; // result of SaveFileDialog
25     string fileName; // name of file containing data
26
27     using (var fileChooser = new SaveFileDialog())
28     {
29         fileChooser.CheckFileExists = false; // let user create file
30         result = fileChooser.ShowDialog();
31         fileName = fileChooser.FileName; // name of file to save data
32     }
33
34     // ensure that user clicked "OK"
35     if (result == DialogResult.OK)
36     {
37         // show error if user specified invalid file
38         if (string.IsNullOrEmpty(fileName))
39         {
40             MessageBox.Show("Invalid File Name", "Error",
41                             MessageBoxButtons.OK, MessageBoxIcon.Error);
42         }
43     }
44 }
```

```
43     else
44     {
45         // save file via FileStream
46         try
47         {
48             // open file with write access
49             var output = new FileStream(fileName,
50                                     FileMode.OpenOrCreate, FileAccess.Write);
51
52             // sets file to where data is written
53             fileWriter = new StreamWriter(output);
54
55             // disable Save button and enable Enter button
56             saveButton.Enabled = false;
57             enterButton.Enabled = true;
58         }
```

```
59         catch (IOException)
60         {
61             // notify user if file does not exist
62             MessageBox.Show("Error opening file", "Error",
63                             MessageBoxButtons.OK, MessageBoxIcon.Error);
64         }
65     }
66 }
67 }
68
```

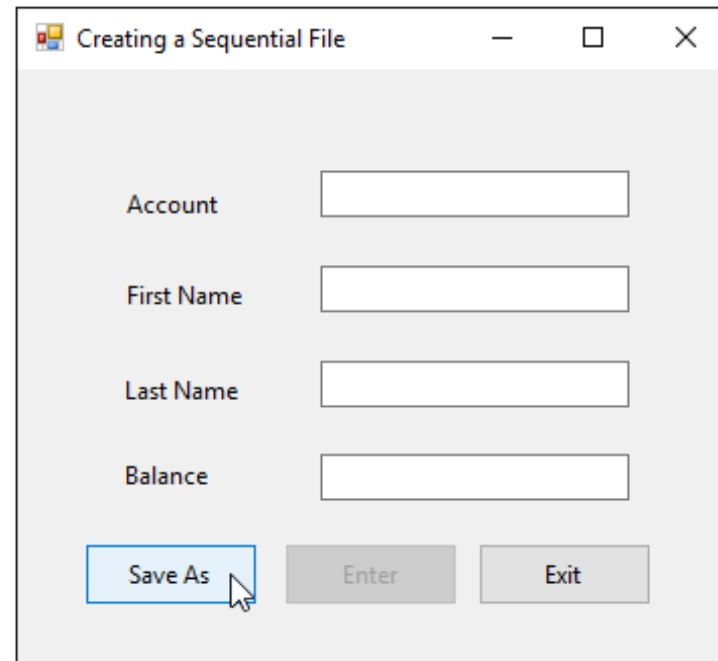
```
69 // handler for enterButton Click
70 private void enterButton_Click(object sender, EventArgs e)
71 {
72     // store TextBox values string array
73     string[] values = GetTextBoxValues();
74
75     // determine whether TextBox account field is empty
76     if (!string.IsNullOrEmpty(values[(int) TextBoxIndices.Account]))
77     {
78         // store TextBox values in Record and output it
79         try
80         {
81             // get account-number value from TextBox
82             int accountNumber =
83                 int.Parse(values[(int) TextBoxIndices.Account]);
84
```

```
85      // determine whether accountNumber is valid
86      if (accountNumber > 0)
87      {
88          // Record containing TextBox values to output
89          var record = new Record(accountNumber,
90                                  values[(int) TextBoxIndices.First],
91                                  values[(int) TextBoxIndices.Last],
92                                  decimal.Parse(values[(int) TextBoxIndices.Balance]));
93
94          // write Record to file, fields separated by commas
95          fileWriter.WriteLine(
96              $"{record.Account},{record.FirstName}," +
97              $"{record.LastName},{record.Balance}");
98      }
99      else
100      {
101          // notify user if invalid account number
102          MessageBox.Show("Invalid Account Number", "Error",
103                          MessageBoxButtons.OK, MessageBoxIcon.Error);
104      }
105  }
```

```
106         catch (IOException)
107         {
108             MessageBox.Show("Error Writing to File", "Error",
109                             MessageBoxButtons.OK, MessageBoxIcon.Error);
110         }
111         catch (FormatException)
112         {
113             MessageBox.Show("Invalid Format", "Error",
114                             MessageBoxButtons.OK, MessageBoxIcon.Error);
115         }
116     }
117
118     ClearTextBoxes(); // clear TextBox values
119 }
120
```

```
121 // handler for exitButton Click
122 private void exitButton_Click(object sender, EventArgs e)
123 {
124     try
125     {
126         fileWriter?.Close(); // close StreamWriter and underlying file
127     }
128     catch (IOException)
129     {
130         MessageBox.Show("Cannot close file", "Error",
131             MessageBoxButtons.OK, MessageBoxIcon.Error);
132     }
133
134     Application.Exit();
135 }
136 }
137 }
```

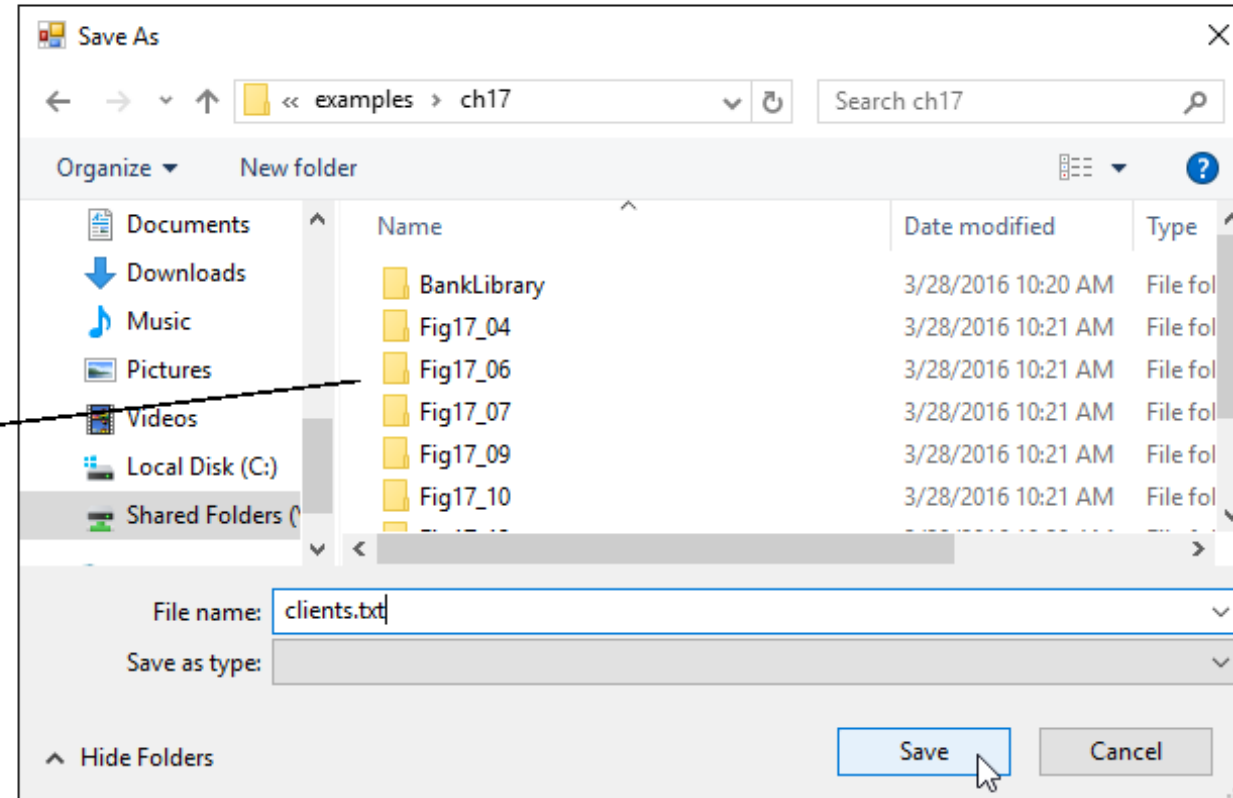
a) BankUI graphical user interface with three additional controls



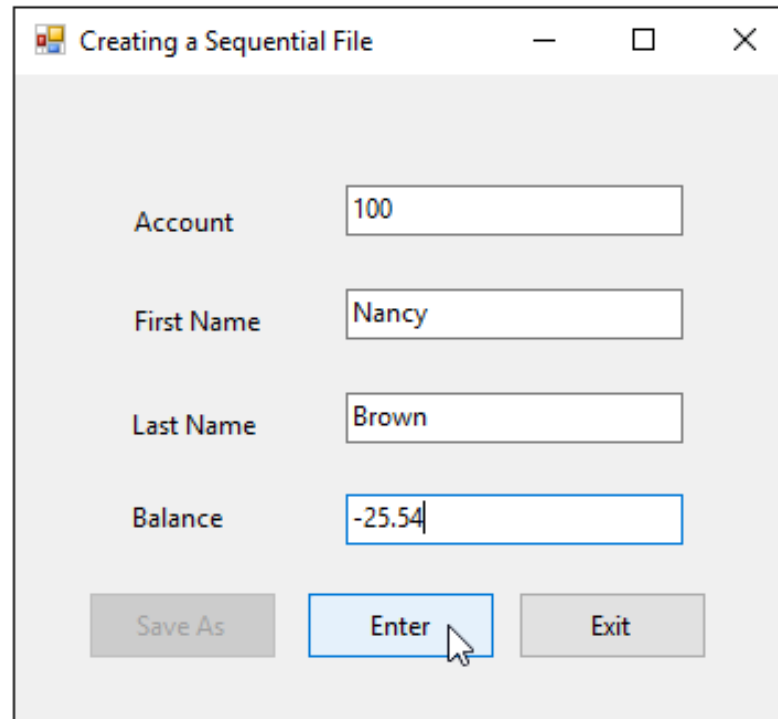
The image shows a graphical user interface window titled "Creating a Sequential File". The window has a standard title bar with a minimize button, a maximize button, and a close button. The main content area is light gray and contains four text input fields, each preceded by a label: "Account", "First Name", "Last Name", and "Balance". Below these fields are three buttons: "Save As" (highlighted with a blue border and a mouse cursor), "Enter" (disabled, gray), and "Exit" (disabled, gray).

b) Save File dialog

Files and directories



c) Account 100,
"Nancy Brown",
saved with a
balance of -25.54



The image shows a Windows-style dialog box titled "Creating a Sequential File". It contains four text input fields with labels to their left: "Account" (containing "100"), "First Name" (containing "Nancy"), "Last Name" (containing "Brown"), and "Balance" (containing "-25.54"). At the bottom of the dialog are three buttons: "Save As" (disabled), "Enter" (active, with a mouse cursor pointing at it), and "Exit" (disabled).

Field Label	Value
Account	100
First Name	Nancy
Last Name	Brown
Balance	-25.54

Buttons: Save As, Enter, Exit

Creating a Sequential-Access Text File (cont.)

Sample Data

- ▶ To test the program, we entered information for the accounts shown in the Table.
- ▶ The program does not depict how the data records are stored in the file.
- ▶ To verify that the file has been created successfully, we create a program in the next section to read and display the file.
- ▶ Since this is a text file, you can actually open it in any text editor to see its contents.

Account number	First name	Last name	Balance
100	Nancy	Brown	-25.54
200	Stacey	Dunn	314.33
300	Doug	Barker	0.00
400	Dave	Smith	258.34
500	Sam	Stone	34.98

Reading Data from a Sequential-Access Text File

- ▶ Class `ReadSequentialAccessFileForm` reads records from the file created by the program, then displays the contents of each record.
- ▶ Much of the code in this example is similar to that program, so we discuss only the unique aspects of the app.

```
1  // Fig. 17.6: ReadSequentialAccessFileForm.cs
2  // Reading a sequential-access file.
3  using System;
4  using System.Windows.Forms;
5  using System.IO;
6  using BankLibrary;
7
8  namespace ReadSequentialAccessFile
9  {
10     public partial class ReadSequentialAccessFileForm : BankUIForm
11     {
12         private StreamReader fileReader; // reads data from a text file
13
14         // parameterless constructor
15         public ReadSequentialAccessFileForm()
16         {
17             InitializeComponent();
18         }
19     }
```

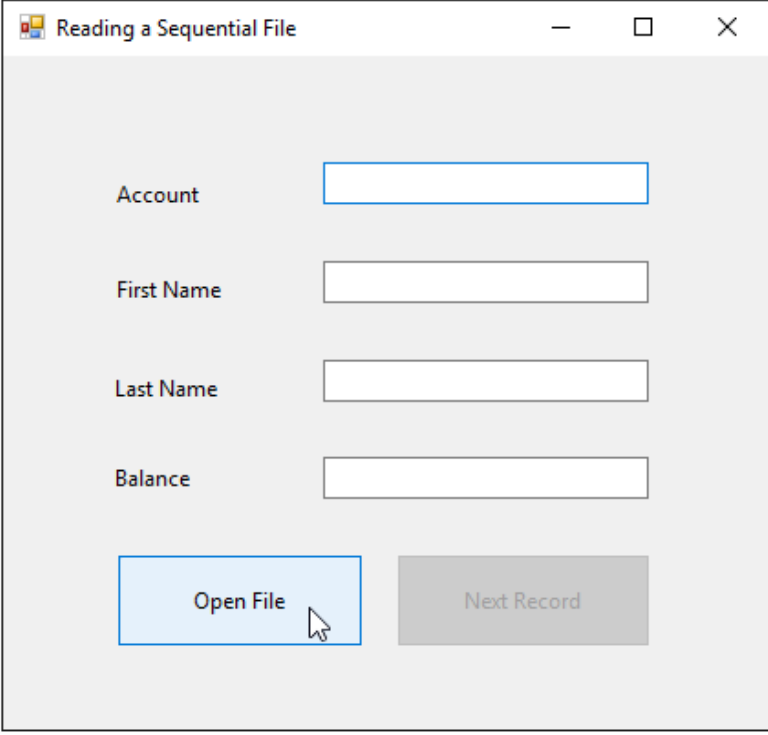
```
20 // invoked when user clicks the Open button
21 private void openButton_Click(object sender, EventArgs e)
22 {
23     // create and show dialog box enabling user to open file
24     DialogResult result; // result of OpenFileDialog
25     string fileName; // name of file containing data
26
27     using (OpenFileDialog fileChooser = new OpenFileDialog())
28     {
29         result = fileChooser.ShowDialog();
30         fileName = fileChooser.FileName; // get specified name
31     }
32
33     // ensure that user clicked "OK"
34     if (result == DialogResult.OK)
35     {
36         ClearTextBoxes();
37
38         // show error if user specified invalid file
39         if (string.IsNullOrEmpty(fileName))
40         {
41             MessageBox.Show("Invalid File Name", "Error",
42                             MessageBoxButtons.OK, MessageBoxIcon.Error);
43         }
44     }
45 }
```

```
44         else
45         {
46             try
47             {
48                 // create FileStream to obtain read access to file
49                 FileStream input = new FileStream(
50                     fileName, FileMode.Open, FileAccess.Read);
51
52                 // set file from where data is read
53                 fileReader = new StreamReader(input);
54
55                 openButton.Enabled = false; // disable Open File button
56                 nextButton.Enabled = true; // enable Next Record button
57             }
58             catch (IOException)
59             {
60                 MessageBox.Show("Error reading from file",
61                     "File Error", MessageBoxButtons.OK,
62                     MessageBoxIcon.Error);
63             }
64         }
65     }
66 }
67
```

```
68 // invoked when user clicks Next button
69 private void nextButton_Click(object sender, EventArgs e)
70 {
71     try
72     {
73         // get next record available in file
74         var inputRecord = fileReader.ReadLine();
75
76         if (inputRecord != null)
77         {
78             string[] inputFields = inputRecord.Split(',');
79
80             // copy string-array values to TextBox values
81             SetTextBoxValues(inputFields);
82         }
83     else
84     {
85         // close StreamReader and underlying file
86         fileReader.Close();
87         openButton.Enabled = true; // enable Open File button
88         nextButton.Enabled = false; // disable Next Record button
89         ClearTextBoxes();
```

```
90
91     // notify user if no records in file
92     MessageBox.Show("No more records in file", string.Empty,
93         MessageBoxButtons.OK, MessageBoxIcon.Information);
94     }
95 }
96 catch (IOException)
97 {
98     MessageBox.Show("Error Reading from File", "Error",
99         MessageBoxButtons.OK, MessageBoxIcon.Error);
100 }
101 }
102 }
103 }
```

a) BankUI graphical user interface with an Open File button



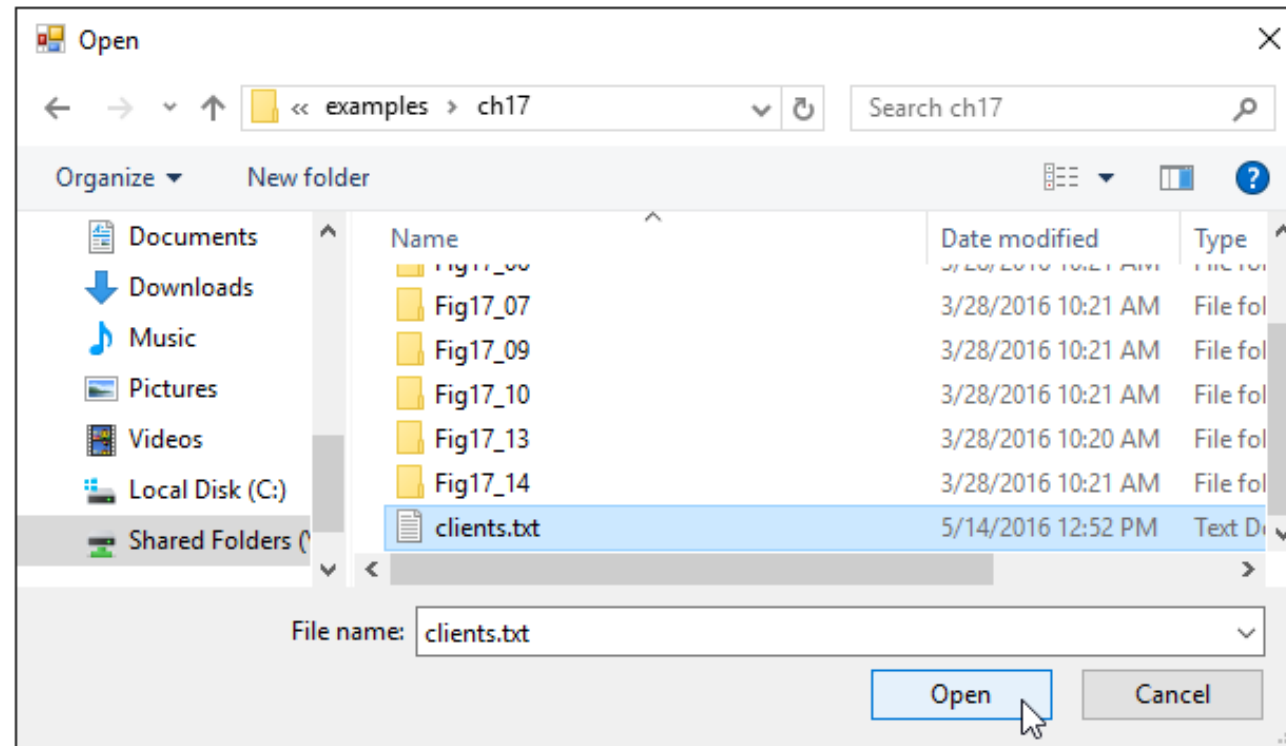
The image shows a graphical user interface window titled "Reading a Sequential File". The window has a standard title bar with a minimize button, a maximize button, and a close button. The main content area is light gray and contains four text input fields stacked vertically. To the left of each input field is a label: "Account", "First Name", "Last Name", and "Balance". Below the input fields are two buttons. The first button is light blue with the text "Open File" and a mouse cursor icon pointing at it. The second button is gray with the text "Next Record".

Field Label	Input Type
Account	Text Input
First Name	Text Input
Last Name	Text Input
Balance	Text Input

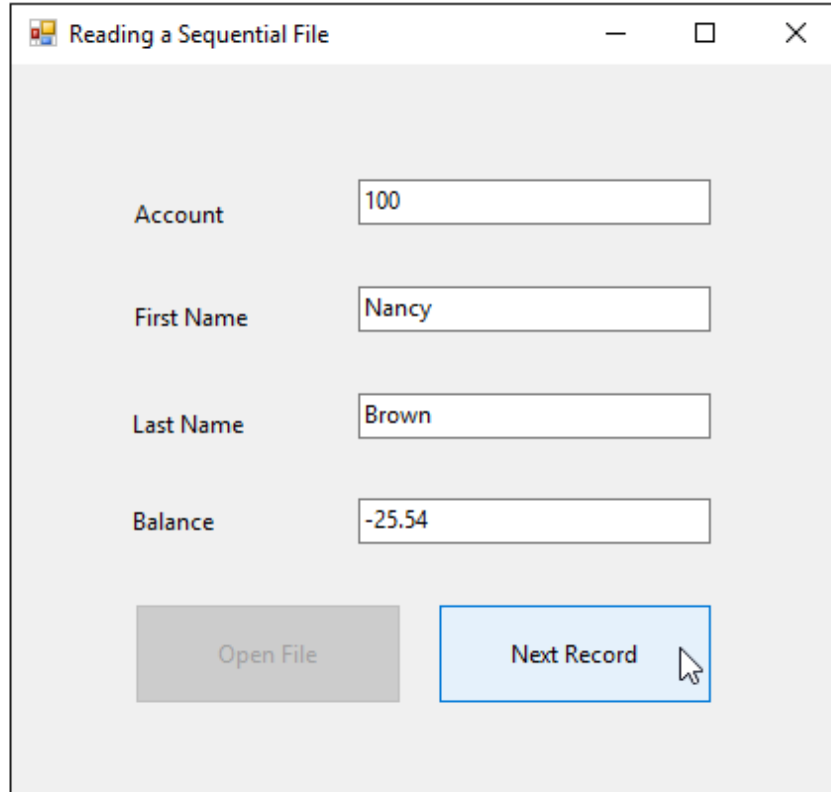
Buttons:

- Open File (Active)
- Next Record (Disabled)

b) OpenFileDialog window



c) Reading account 100



Reading a Sequential File

Account 100

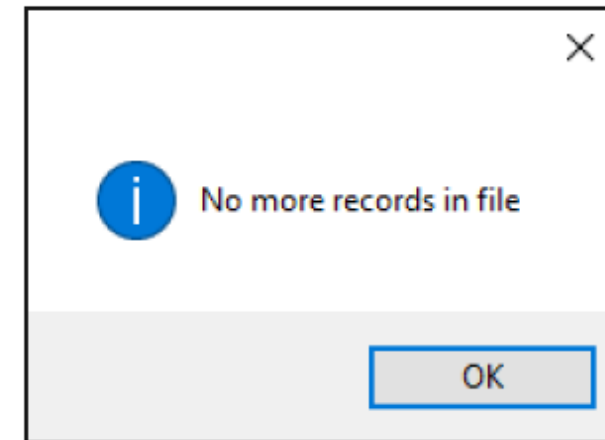
First Name Nancy

Last Name Brown

Balance -25.54

Open File Next Record

d) User is shown a messagebox when all records have been read



Serialization

- ▶ In the prior examples, Record was used to aggregate the information for one record.
- ▶ When the instance variables for a Record were output to a disk file, certain information was lost, such as the type of each value.
- ▶ For instance, if the value "3" is read from a file, there's no way to tell if the value came from an `int`, a `string` or a `decimal`.

Serialization (cont.)

- ▶ We have only data, not type information, on disk.
- ▶ If the program that's going to read this data “knows” what object type the data corresponds to, then the data can be read directly into objects of that type.
- ▶ For example, we know that we are inputting an `int` (the account number), the first and last name `strings` and a `decimal` (the balance).
- ▶ We also know that these values are separated by commas, with only one record on each line.
- ▶ So, we are able to parse the `strings` and convert the account number to an `int` and the balance to a `decimal`.

Serialization (cont.)

- ▶ Sometimes it would be easier to read or write entire objects.
- ▶ C# provides such a mechanism, called object serialization.
- ▶ A serialized object is an object represented as a sequence of bytes that includes the object's data, as well as information about the object's type and the types of data stored in the object.
- ▶ After a serialized object has been written to a file, it can be read from the file and deserialized—that is, the type information and bytes that represent the object and its data can be used to recreate the object in memory.

Serialization (cont.)

- ▶ Class `BinaryFormatter` (namespace `System.Runtime.Serialization.Formatters.Binary`) enables entire objects to be written to or read from a stream.
- ▶ `BinaryFormatter` method `Serialize` writes an object's representation to a file.
- ▶ `BinaryFormatter` method `Deserialize` reads this representation from a file and reconstructs the original object.
- ▶ Both methods throw a `SerializationException` if an error occurs during serialization or deserialization.
- ▶ Both methods require a `Stream` object (e.g., the `FileStream`) as a parameter so that the `BinaryFormatter` can access the correct stream.

Serialization (cont.)

- ▶ Next, we create and manipulate sequential-access files using object serialization.
- ▶ Object serialization is performed with byte-based streams, so the sequential files created and manipulated will be binary files.
- ▶ Binary files are not human readable.
- ▶ For this reason, we write a separate app that reads and displays serialized objects.

Creating a Sequential-Access File Using Object Serialization

- ▶ We begin by creating and writing serialized objects to a sequential-access file.
- ▶ In this section, we reuse much of the code from earlier sections, so we focus only on the new features.

Creating a Sequential-Access File Using Object Serialization (cont.)

Defining the RecordSerializable Class

- ▶ Class `RecordSerializable` (part of the `BankLibrary` project) is marked with the `[Serializable]` attribute, which indicates to the CLR that `RecordSerializable` objects can be serialized.
- ▶ Classes that represent serializable types must include this attribute in their declarations or must implement interface `ISerializable`.

```
1  // Fig. 17.8: RecordSerializable.cs
2  // Serializable class that represents a data record.
3  using System;
4
5  namespace BankLibrary
6  {
7      [Serializable]
8      public class RecordSerializable
9      {
10         public int Account { get; set; }
11         public string FirstName { get; set; }
12         public string LastName { get; set; }
13         public decimal Balance { get; set; }
14
15         // default constructor sets members to default values
16         public RecordSerializable()
17             : this(0, string.Empty, string.Empty, 0M) {}
18     }
```

```
19      // overloaded constructor sets members to parameter values
20      public RecordSerializable(int account, string firstName,
21          string lastName, decimal balance)
22      {
23          Account = account;
24          FirstName = firstName;
25          LastName = lastName;
26          Balance = balance;
27      }
28  }
29 }
```

Creating a Sequential-Access File Using Object Serialization (cont.)

- ▶ In a class that's marked with the `[Serializable]` attribute or that implements interface `ISerializable`, you must ensure that every instance variable of the class is also serializable.
- ▶ All simple-type variables and strings are serializable.
- ▶ For variables of reference types, you must check the class declaration (and possibly its base classes) to ensure that the type is serializable.
- ▶ By default, array objects are serializable.
- ▶ However, if the array contains references to other objects, those objects may or may not be serializable.

Creating a Sequential-Access File Using Object Serialization (cont.)

Using a Serialization Stream to Create an Output File

- ▶ Next, we'll create a sequential-access file with serialization.
- ▶ To test this program, we used the sample data from the table to create a file named `clients.ser`.

Creating a Sequential-Access File Using Object Serialization (cont.)

- ▶ The program does not show how the data records actually appear in the file.
- ▶ Remember that we are now using binary files, which are not human readable.
- ▶ To verify that the file was created successfully, the next section presents a program to read the file's contents.

```
1  // Fig. 17.9: CreateFileForm.cs
2  // Creating a sequential-access file using serialization.
3  using System;
4  using System.Windows.Forms;
5  using System.IO;
6  using System.Runtime.Serialization.Formatters.Binary;
7  using System.Runtime.Serialization;
8  using BankLibrary;
9
10 namespace CreateFile
11 {
12     public partial class CreateFileForm : BankUIForm
13     {
14         // object for serializing RecordSerializables in binary format
15         private BinaryFormatter formatter = new BinaryFormatter();
16         private FileStream output; // stream for writing to a file
17
18         // parameterless constructor
19         public CreateFileForm()
20         {
21             InitializeComponent();
22         }
23     }
24 }
```

```
23
24 // handler for saveButton_Click
25 private void saveButton_Click(object sender, EventArgs e)
26 {
27     // create and show dialog box enabling user to save file
28     DialogResult result;
29     string fileName; // name of file to save data
30
31     using (SaveFileDialog fileChooser = new SaveFileDialog())
32     {
33         fileChooser.CheckFileExists = false; // let user create file
34
35         // retrieve the result of the dialog box
36         result = fileChooser.ShowDialog();
37         fileName = fileChooser.FileName; // get specified file name
38     }
39
```

```
40 // ensure that user clicked "OK"
41 if (result == DialogResult.OK)
42 {
43     // show error if user specified invalid file
44     if (string.IsNullOrEmpty(fileName))
45     {
46         MessageBox.Show("Invalid File Name", "Error",
47             MessageBoxButtons.OK, MessageBoxIcon.Error);
48     }
49     else
50     {
51         // save file via FileStream if user specified valid file
52         try
53         {
54             // open file with write access
55             output = new FileStream(fileName,
56                 FileMode.OpenOrCreate, FileAccess.Write);
57
58             // disable Save button and enable Enter button
59             saveButton.Enabled = false;
60             enterButton.Enabled = true;
61         }
```

```
62         catch (IOException)
63         {
64             // notify user if file could not be opened
65             MessageBox.Show("Error opening file", "Error",
66                             MessageBoxButtons.OK, MessageBoxIcon.Error);
67         }
68     }
69 }
70 }
71
72 // handler for enterButton Click
73 private void enterButton_Click(object sender, EventArgs e)
74 {
75     // store TextBox values string array
76     string[] values = GetTextBoxValues();
77
78     // determine whether TextBox account field is empty
79     if (!string.IsNullOrEmpty(values[(int) TextBoxIndices.Account]))
80     {
81         // store TextBox values in RecordSerializable and serialize it
82         try
83         {
```

```
84         // get account-number value from TextBox
85         int accountNumber = int.Parse(
86             values[(int) TextBoxIndices.Account]);
87
88         // determine whether accountNumber is valid
89         if (accountNumber > 0)
90         {
91             // RecordSerializable to serialize
92             var record = new RecordSerializable(accountNumber,
93                 values[(int) TextBoxIndices.First],
94                 values[(int) TextBoxIndices.Last],
95                 decimal.Parse(values[(int) TextBoxIndices.Balance]));
96
97             // write Record to FileStream (serialize object)
98             formatter.Serialize(output, record);
99         }
100        else
101        {
102            // notify user if invalid account number
103            MessageBox.Show("Invalid Account Number", "Error",
104                MessageBoxButtons.OK, MessageBoxIcon.Error);
105        }
106    }
```

```
107         catch (SerializationException)
108         {
109             MessageBox.Show("Error Writing to File", "Error",
110                 MessageBoxButtons.OK, MessageBoxIcon.Error);
111         }
112         catch (FormatException)
113         {
114             MessageBox.Show("Invalid Format", "Error",
115                 MessageBoxButtons.OK, MessageBoxIcon.Error);
116         }
117     }
118
119     ClearTextBoxes(); // clear TextBox values
120 }
121
```

```
122    // handler for exitButton Click
123    private void exitButton_Click(object sender, EventArgs e)
124    {
125        // close file
126        try
127        {
128            output?.Close(); // close FileStream
129        }
130        catch (IOException)
131        {
132            MessageBox.Show("Cannot close file", "Error",
133                            MessageBoxButtons.OK, MessageBoxIcon.Error);
134        }
135
136        Application.Exit();
137    }
138 }
139 }
```

Reading and Deserializing Data from a Binary File

- ▶ The preceding section showed how to create a sequential-access file using object serialization.
- ▶ In this section, we discuss how to read serialized objects sequentially from a file.

Reading and Deserializing Data from a Binary File (cont.)

- ▶ The program reads objects from a file in event handler `nextButton_Click`.
- ▶ We use method `Deserialize` to read the data.
- ▶ Note that we cast the result of `Deserialize` to type `RecordSerializable`—this cast is necessary, because `Deserialize` returns a reference of type `object` and we need to access properties that belong to class `RecordSerializable`.
- ▶ If an error occurs during deserialization or the end of the file is reached, a `SerializationException` is thrown, and the `FileStream` object is closed.

```
1  // Fig. 17.10: ReadSequentialAccessFileForm.cs
2  // Reading a sequential-access file using deserialization.
3  using System;
4  using System.Windows.Forms;
5  using System.IO;
6  using System.Runtime.Serialization.Formatters.Binary;
7  using System.Runtime.Serialization;
8  using BankLibrary;
9
10 namespace ReadSequentialAccessFile
11 {
12     public partial class ReadSequentialAccessFileForm : BankUIForm
13     {
14         // object for deserializing RecordSerializable in binary format
15         private BinaryFormatter reader = new BinaryFormatter();
16         private FileStream input; // stream for reading from a file
17
18         // parameterless constructor
19         public ReadSequentialAccessFileForm()
20         {
21             InitializeComponent();
22         }
```

```
23
24 // invoked when user clicks the Open button
25 private void openButton_Click(object sender, EventArgs e)
26 {
27     // create and show dialog box enabling user to open file
28     DialogResult result; // result of OpenFileDialog
29     string fileName; // name of file containing data
30
31     using (OpenFileDialog fileChooser = new OpenFileDialog())
32     {
33         result = fileChooser.ShowDialog();
34         fileName = fileChooser.FileName; // get specified name
35     }
36
```

```
37         // ensure that user clicked "OK"
38         if (result == DialogResult.OK)
39         {
40             ClearTextBoxes();
41
42             // show error if user specified invalid file
43             if (string.IsNullOrEmpty(fileName))
44             {
45                 MessageBox.Show("Invalid File Name", "Error",
46                     MessageBoxButtons.OK, MessageBoxIcon.Error);
47             }
48             else
49             {
50                 // create FileStream to obtain read access to file
51                 input = new FileStream(
52                     fileName, FileMode.Open, FileAccess.Read);
53
54                 openButton.Enabled = false; // disable Open File button
55                 nextButton.Enabled = true;  // enable Next Record button
56             }
57         }
58     }
59
```

```
60      // invoked when user clicks Next button
61      private void nextButton_Click(object sender, EventArgs e)
62      {
63          // deserialize RecordSerializable and store data in TextBoxes
64          try
65          {
66              // get next RecordSerializable available in file
67              RecordSerializable record =
68                  (RecordSerializable) reader.Deserialize(input);
69
70              // store RecordSerializable values in temporary string array
71              var values = new string[] {
72                  record.Account.ToString(),
73                  record.FirstName.ToString(),
74                  record.LastName.ToString(),
75                  record.Balance.ToString()
76              };
77
78              // copy string-array values to TextBox values
79              SetTextBoxValues(values);
80          }
```

```
81         catch (SerializationException)
82         {
83             input?.Close(); // close FileStream
84             openButton.Enabled = true; // enable Open File button
85             nextButton.Enabled = false; // disable Next Record button
86
87             ClearTextBoxes();
88
89             // notify user if no RecordSerializables in file
90             MessageBox.Show("No more records in file", string.Empty,
91                             MessageBoxButtons.OK, MessageBoxIcon.Information);
92         }
93     }
94 }
95 }
```
